Lab 2: Network Delay, Packet Loss, and Throughput with CML and Wireshark

1. Background

In packet-switched networks, the performance of data transmission is governed by three fundamental metrics:

- **Delay (Latency)**: The time it takes for a packet to travel from the source to the destination. The end-to-end delay is the sum of processing delay, queuing delay, transmission delay, and propagation delay.
- **Packet Loss**: The percentage of packets that fail to reach their destination. This typically occurs when a router's buffer overflows due to congestion, causing the router to drop incoming packets.
- **Throughput**: The rate at which data is successfully delivered over a communication channel. Throughput is limited by the bottleneck link along the end-to-end path.

Cisco Modeling Labs (CML) provides **Link Condition** capabilities on links. This allows you to apply simulated impairments (latency, packet loss, and bandwidth limits) directly to the link connecting two nodes to observe and analyze network behavior under different conditions.

---

2. Objective / Purpose

The objective of this lab is to configure, measure, and analyze the direct impacts of network delay, packet loss, and bandwidth constraints using a simulated client-server topology in CML.

Specifically, you will:

- Set up a direct link between two **Alpine Linux** hosts representing a Client and a Server.
- Configure link impairments (latency, packet loss, and bandwidth limits) in CML.
- Use `ping` to measure round-trip time (RTT) and packet loss rates.
- Use BusyBox `httpd` on the server and `wget` on the client to measure throughput under different bandwidth constraints.
- Capture the traffic in Wireshark to observe TCP retransmissions and packet drops.

---

3. CML Topology and Configuration Procedure

```
graph LR
    H1["Client Node (H1)<br>OS: Alpine Linux<br>eth0: 192.168.1.1/24"] <--> |"eth0 <--> eth0<br>
[CML Link Condition Applied]"| H2["Server Node (H2)<br>OS: Alpine Linux<br>eth0:
192.168.1.2/24<br>Services: busybox-extras httpd (Port 80)"]

    classDef alpine fill:#d4f1f9,stroke:#007b8b,stroke-width:2px;
    class H1,H2 alpine;
```

Step 1: Deploying the Nodes

1. Log in to the CML UI and create a new lab called `Lab_02_Delay_Loss_Throughput`.
2. Drag two **Alpine Linux** nodes onto the workspace.
3. Rename the first node `<initials>-H1` (this will be the Client) and the second node `<initials>-H2` (this will be the Server).
4. Create a link connecting the two nodes (`eth0` on H1 to `eth0` on H2).

Step 2: Configuring the Client and Server

You will use CML's **Edit Config** tab to inject configuration scripts that execute automatically when the nodes boot.

1. Client Host (H1) Configuration

- Click on the H1 client node in the workspace.
- In the bottom pane, navigate to the **Edit Config** tab.
- Enter the following shell script to configure its static IP address, start the interface, and signal CML that the boot process is complete:

```
#!/bin/sh
echo "=== Starting Alpine H1 Boot Script ===" >/dev/console

# Configure network interface eth0
echo "Configuring eth0 static IP..." >/dev/console
ip addr add 192.168.1.1/24 dev eth0
ip link set eth0 up
ip addr show dev eth0 >/dev/console

# Signal boot completion to CML
echo "READY" >/dev/console
exit 0
```

2. Server Host (H2) Configuration

- Click on the H2 server node in the workspace.
- Navigate to the **Edit Config** tab.
- Enter the following shell script to configure its static IP address, create a large test file, start the built-in lightweight HTTP server, and notify CML that it is ready:

```
#!/bin/sh
echo "=== Starting Alpine H2 Boot Script ===" >/dev/console

# Configure network interface eth0
echo "Configuring eth0 static IP..." >/dev/console
ip addr add 192.168.1.2/24 dev eth0
ip link set eth0 up
ip addr show dev eth0 >/dev/console

# Wait for network interface initialization
sleep 1

# Create directory for web documents
echo "Creating web document directory..." >/dev/console
mkdir -p /var/www

# Create a large file (~10MB) for throughput measurement tests
echo "Creating 10MB test file (large.bin)..." >/dev/console
dd if=/dev/urandom of=/var/www/large.bin bs=1M count=10 >/dev/console 2>&1

# Start BusyBox lightweight HTTP server on port 80
echo "Starting BusyBox HTTP server..." >/dev/console
```

```
busybox-extras httpd -p 80 -h /var/www

# Notify CML that the boot process has completed
echo "READY" >/dev/console
exit 0
```

Step 3: Starting the Nodes

1. Click the **Start Lab** button (play icon) at the top of the workspace to boot up both nodes.
2. Wait for the nodes to turn green, indicating they are fully active.

---

4. Lab Execution and Impairment Testing

Part A: Measuring Link Delay (Latency)

1. Verify connectivity: Click on the H1 Client node. In the bottom pane, select the **Console** tab and press **Enter** to get a prompt. Ping the H2 server:

   ```
   ping -c 5 192.168.1.2
   ```

   Record the minimum, average, and maximum Round-Trip Time (RTT) from the summary.

2. Configure Link Latency:

   - Click on the **link** connecting H1 and H2 in the CML workspace.
   - In the bottom pane, navigate to the **Link Condition** (or **Link Effects**) tab.
   - Set the **Latency (Delay)** to `50` ms. Click **Save** or **Update**.
   - *Note: CML applies latency on a per-direction basis. A setting of 50ms adds 50ms of delay to packets traveling from H1 -> H2, and 50ms of delay to packets traveling from H2 -> H1.*

3. Run the ping test again:

   ```
   ping -c 10 192.168.1.2
   ```

   Record the new minimum, average, and maximum RTT. How does it compare to the configured link latency?

4. Increase Latency:

   - Click on the link again and change the **Latency** to `150` ms. Save the changes.
   - Run the ping test once more and record the RTT summary.
   - Reset the link **Latency** back to `0` ms when done.

Part B: Measuring Packet Loss

1. **Start Packet Capture**:

   - Click on the link between H1 and H2.
   - Navigate to the **Packet Capture** tab and click **Start**.

2. Configure Link Packet Loss:

   - Click on the link. Under the **Link Condition** tab, set the **Packet Loss** to `10` %. Click **Save**.

3. Run the Ping Test:

- In the H1 Client console, send 50 ping requests:

  ```
  ping -c 50 192.168.1.2
  ```

  Record the percentage of packet loss reported in the ping summary.

4. Generate TCP Traffic under Loss:

   - Run a download test of the large file:

     ```
     wget -O /dev/null http://192.168.1.2/large.bin
     ```

     Note if there is any visible lag or drop in speed compared to a clean link.

5. **Download the Capture**:

   - Go to the link's **Packet Capture** tab, click **Stop**, and then click **Download** to save the `.pcap` file.
   - Reset the link **Packet Loss** back to `0` %.

Part C: Measuring Throughput (Bandwidth Constraints)

1. Configure Link Bandwidth:

   - Click on the link. Under the **Link Condition** tab, set the **Bandwidth** (or throughput limit) to `10` Mbps (or `10000` kbps depending on CML version). Click **Save**.

2. Run the Download Test:

   - In H1's console, download the large file and observe the average transfer rate reported by `wget`:

     ```
     wget -O /dev/null http://192.168.1.2/large.bin
     ```

     Record the average transfer speed (in MB/s or KB/s).

3. Restrict Link Bandwidth:

   - Click on the link and set the **Bandwidth** limit to `500` kbps. Click **Save**.
   - Run the download test again:

     ```
     wget -O /dev/null http://192.168.1.2/large.bin
     ```

     Record the new average transfer speed. Note how the progress bar responds.

4. Reset link impairments back to default (Bandwidth unlimited / default, Latency 0ms, Loss 0%).

---

5. Wireshark Analysis and Questions

Open your downloaded packet capture from **Part B** in Wireshark and answer the following questions.

> ***IMPORTANT - Annotation Requirement****: For questions 3 and 5, you must include a screenshot of the relevant packets in Wireshark and* ***annotate it*** *(e.g., using arrows, circles, or text callouts) to clearly point out the packets, fields, or sequence numbers you are describing. Handing in unannotated screenshots will result in a point deduction.*

5.1 Latency and RTT

1. What is the relationship between the configured one-way link latency in CML and the actual Round-Trip Time (RTT) observed in your ping results? Why does this relationship exist?
2. In Part A, did the RTT remain perfectly constant, or was there variation (jitter)? Explain what factors could introduce RTT variation in a virtual/emulated environment.

5.2 Packet Loss Analysis

3. Filter your capture in Wireshark for `icmp`. Look at the sequence numbers of the ICMP Echo Requests (`icmp.seq`). Do you see gaps in the sequence numbers? Find one missing request and provide an **annotated screenshot** pointing out the packet numbers immediately before and after the gap.
4. When an ICMP Echo Request is lost due to packet loss, does the server send an Echo Reply? Explain using your Wireshark observations.
5. Filter your capture for TCP retransmissions: `tcp.analysis.retransmission`. Provide an **annotated screenshot** showing a section of the packet list. Circle or highlight the retransmitted packets and point out the duplicate ACKs or timeouts that triggered them. Explain why TCP retransmits packets and the mechanism it used in this instance to detect the loss.
6. How does packet loss affect the overall transmission time of a TCP stream compared to a lossless link? Refer to your `wget` results.

5.3 Throughput Analysis

7. Compare your download speeds from Part C. When the bandwidth limit was `10 Mbps`, what was the actual download speed in MB/s? When the limit was `500 kbps`, what was the speed? (Convert your download speeds to bits per second to compare them directly to the link limit: $Speed\ in\ bps = Speed\ in\ Bytes/sec \times 8$).
8. Explain the difference between **throughput** and **bandwidth**. Which one represents the theoretical maximum capacity of the link, and which one represents the actual observed rate of successful data delivery?

---

6. What to Hand In

Submit a lab report containing:

1. A screenshot of your CML topology showing the connected H1 and H2 hosts.
2. Console output screenshots of:
   - The ping RTT summaries for 0ms, 50ms, and 150ms delays on H1.
   - The ping summary showing the packet loss percentage on H1.
   - The wget download speed outputs for 10 Mbps and 500 kbps bandwidths.
3. The downloaded `.pcap` capture file from Part B.
4. Annotated screenshots from Wireshark showing:
   - The gap in ICMP sequence numbers (from Question 3).
   - The TCP retransmissions and their duplicate ACK / timeout triggers (from Question 5).
5. Your written answers to the 8 analysis questions in Section 5.